

Impacto da Política de Substituição de Cache na Eficiência de Canais Laterais: LRU versus Tree-PLRU

Felipe Gabriel de Carvalho¹, Felipe Silva Barros¹, Gabriel Henrique de Morais¹
Gustavo Henrique dos Santos Dias¹, Thiago Henrique Gomes Feliciano¹

¹Instituto de Ciências Exatas e Informática (ICEI)
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Campus Coração Eucarístico – Belo Horizonte – MG – Brasil

Abstract. *Cache replacement policies, although designed with performance in mind, can expose state metadata that is exploitable as a side channel. Xiong et al. (2021) showed that LRU and PLRU states can be used as a timing channel even on cache hits, but their evaluation focused mainly on the L1 cache. This work compares LRU and Tree-PLRU in the L2 cache through gem5 simulation, evaluating their impact on memory hierarchy performance through cache miss rate, latency and execution behavior. For the workload used, both policies produced identical results, since the L2 miss rate was 99.5% and the absence of reuse makes the victim choice irrelevant. The results therefore do not indicate a performance difference between the two policies under the evaluated workload. The work documents the simulation environment and discusses the workload requirements needed to differentiate replacement policies.*

Resumo. *Políticas de substituição de cache, embora projetadas com foco em desempenho, podem expor metadados de estado exploráveis como canais laterais. [Xiong et al. 2021] demonstraram que o estado das políticas LRU e PLRU pode ser utilizado como canal de temporização mesmo em cache hits, mas sua avaliação concentrou-se principalmente no cache L1. Este trabalho compara, por meio de simulação no gem5, as políticas LRU e Tree-PLRU na cache L2, avaliando seu impacto no desempenho da hierarquia de memória por meio da taxa de cache miss, latência e comportamento da execução. Para o workload utilizado, as duas políticas produziram resultados idênticos, pois a taxa de miss da L2 foi de 99,5% e a ausência de reuso torna irrelevante a escolha da vítima. Os resultados, portanto, não indicam diferença de desempenho entre as duas políticas para o workload avaliado. O trabalho documenta o ambiente de simulação e discute os requisitos de workload necessários para diferenciar políticas de substituição.*

1. Introdução

A hierarquia de memória desempenha um papel importante no desempenho dos processadores modernos, permitindo reduzir o tempo necessário para o acesso a dados utilizados com frequência. Dentro dessa hierarquia, as memórias cache utilizam diferentes políticas de substituição para determinar quais linhas devem ser removidas quando novos dados precisam ser armazenados. Embora essas políticas sejam projetadas principalmente com foco em desempenho, seu estado interno também pode apresentar implicações para a segurança dos sistemas.

Ataques de canal lateral exploram características do comportamento do hardware para obter informações sem acessar diretamente os dados da vítima. Nesse contexto, Xiong et al. (2021) demonstraram que o estado das políticas de substituição LRU e PLRU pode ser utilizado como um canal de temporização e comunicação entre processos. Um aspecto relevante do trabalho é que o vazamento pode ocorrer mesmo quando os acessos resultam em cache hits, indicando que não apenas o conteúdo armazenado na cache, mas também seus metadados e mecanismos internos de substituição, podem representar uma superfície de ataque [Xiong et al. 2021].

Diferentes políticas de substituição, entretanto, mantêm e atualizam essas informações de maneiras distintas. A política LRU (*Least Recently Used*) busca identificar a linha menos recentemente utilizada, enquanto políticas aproximadas, como Tree-PLRU (*Tree-based Pseudo-LRU*), utilizam estruturas simplificadas para representar a ordem de utilização das linhas. Essas diferenças podem influenciar tanto o comportamento de um canal lateral baseado no estado da cache quanto o desempenho da hierarquia de memória.

Hipotetiza-se que a política Tree-PLRU, por representar de forma aproximada e simplificada a ordem de uso das linhas de cache, pode apresentar comportamento de desempenho diferente da política LRU, em razão da menor complexidade de atualização de seu estado interno.

Dessa forma, este trabalho tem como objetivo investigar o impacto da política de substituição na hierarquia de memória, comparando as políticas LRU e Tree-PLRU na cache L2 em um ambiente controlado de simulação. Foram coletadas métricas de desempenho, como cache miss rate, latência média de acesso, número de substituições e tempo de execução.

Os resultados mostram que, para o workload avaliado, a Tree-PLRU não altera o desempenho em relação à LRU, pois a alta taxa de miss da L2 torna a escolha da vítima irrelevante. Esse resultado destaca a importância da escolha do workload em estudos de políticas de substituição. O restante do artigo está organizado da seguinte forma: a Seção 2 discute os trabalhos correlatos, a Seção 3 apresenta a metodologia, a Seção 4 descreve o ambiente experimental, a Seção 5 apresenta e discute os resultados e a Seção 6 apresenta as conclusões e os trabalhos futuros.

2. Discussão dos Trabalhos Correlatos

2.1. Leaking Information Through Cache LRU States in Commercial Processors and Secure Caches

Xiong, Katzenbeisser e Szefer (2021) investigaram o uso do estado das políticas de substituição LRU e PLRU como canal de temporização e de comunicação entre processos. O principal diferencial do trabalho é demonstrar que o vazamento pode ocorrer mesmo quando os acessos resultam em cache hits, explorando as alterações no estado interno da política de substituição. Os autores realizaram experimentos em processadores Intel e AMD e também analisaram mecanismos de proteção, como Partition-Locked e Random Fill [Xiong et al. 2021].

Apesar de sua contribuição, o estudo apresenta limitações importantes. A avaliação experimental concentra-se principalmente no cache L1, enquanto os níveis L2 e LLC são discutidos, mas não avaliados experimentalmente de forma abrangente. Além disso,

parte dos mecanismos de defesa analisados foi avaliada apenas de forma qualitativa. Essas limitações motivam a investigação de arquiteturas mais recentes e de outros níveis da hierarquia de cache [Xiong et al. 2021].

2.2. Generic and Automated Drive-by GPU Cache Attacks from the Browser

Giner et al. (2024) investigaram ataques à cache L2 de GPUs realizados a partir de navegadores utilizando WebGPU. O trabalho demonstra que ataques de cache podem ser realizados em um ambiente mais restrito do que o tradicional cenário de código nativo, utilizando técnicas como Prime+Probe e mecanismos desenvolvidos para identificar características da cache e construir conjuntos de evasão. Os autores também demonstraram a possibilidade de utilizar essas técnicas em ataques contra informações processadas pela GPU [Giner et al. 2024].

A principal diferença em relação a Xiong et al. (2021) está tanto no ambiente de execução quanto no mecanismo explorado. Enquanto Xiong et al. concentram-se no estado LRU/PLRU e em alterações observáveis mesmo com cache hits, Giner et al. exploram alterações relacionadas à ocupação e à expulsão de linhas da cache. Dessa forma, o trabalho amplia o cenário de ameaça para GPUs e aplicações executadas por meio de navegadores [Xiong et al. 2021, Giner et al. 2024].

2.3. Random and Safe Cache Architecture to Defeat Cache Timing Attacks

Hu e Lee (2024) apresentam a arquitetura Random and Safe Cache (RaS) como uma abordagem defensiva contra ataques de temporização. A proposta busca reduzir a previsibilidade do comportamento da cache por meio da substituição das políticas determinísticas tradicionais por mecanismos de seleção aleatória, além de utilizar um Safe History Buffer. A arquitetura foi avaliada por meio de simulações no gem5 contra diferentes técnicas de ataque, incluindo Spectre, Prime+Probe, Flush+Reload e Evict-Time [Hu and Lee 2024].

A vantagem do trabalho está em abordar diretamente a questão da proteção da cache, enquanto outros estudos concentram-se na identificação ou exploração das vulnerabilidades. Entretanto, uma limitação relevante é que a arquitetura RaS não foi diretamente avaliada contra o canal específico apresentado por Xiong et al. (2021), baseado na observação do estado LRU/PLRU por meio de cache hits. Assim, permanece a questão de até que ponto mecanismos de randomização são capazes de eliminar esse tipo específico de canal [Hu and Lee 2024, Xiong et al. 2021].

2.4. Analysis of Secure Caches Using a Three-Step Model for Timing-Based Attacks

Deng, Xiong e Szefer (2019) propuseram um modelo de três etapas para analisar sistematicamente vulnerabilidades relacionadas a ataques de temporização em caches. A metodologia considera diferentes formas pelas quais um atacante pode modificar o estado da cache e posteriormente observar alterações nesse estado, permitindo identificar possíveis canais de comunicação ou vazamento de informações [Deng et al. 2019].

A principal contribuição do trabalho está, portanto, na criação de uma metodologia sistemática de análise, servindo como base para a identificação de diferentes classes de vulnerabilidades. Entretanto, grande parte da avaliação é baseada em modelos e análises teóricas, apresentando uma validação experimental mais limitada em hardware comercial. Dessa forma, embora seja útil para identificar possíveis canais, a exploração

prática desses canais ainda depende das características específicas de cada arquitetura [Deng et al. 2019].

2.5. Prime+Retouch: When Cache is Locked and Leaked

Lee, Sang e Kim (2024) apresentaram o ataque Prime+Retouch, demonstrando que informações relacionadas aos metadados da política de substituição podem ser exploradas mesmo quando os dados da vítima estão protegidos contra substituição. O ataque foi demonstrado em arquiteturas Intel x86 e Apple M1, mostrando que mecanismos associados à substituição continuam representando uma possível superfície de ataque em arquiteturas modernas [Lee et al. 2024].

Esse trabalho possui relação direta com Xiong et al. (2021), pois ambos demonstram que proteger apenas os dados armazenados na cache pode não ser suficiente. Entretanto, enquanto Xiong et al. exploram diretamente o estado LRU/PLRU como canal de temporização, Lee et al. exploram os metadados relacionados à política de substituição. A utilização de arquiteturas mais recentes também reforça a importância de verificar se vulnerabilidades relacionadas aos mecanismos de substituição permanecem presentes em diferentes gerações de processadores [Xiong et al. 2021, Lee et al. 2024].

2.6. Comparação dos trabalhos e posicionamento da proposta

Os cinco trabalhos apresentam diferentes perspectivas sobre a segurança de caches. Xiong et al. (2021) demonstram experimentalmente a existência de um canal baseado no estado LRU/PLRU; Giner et al. (2024) ampliam o estudo de ataques de cache para GPUs e navegadores; Hu e Lee (2024) apresentam uma arquitetura defensiva baseada em randomização; Deng et al. (2019) propõem uma metodologia sistemática para identificação de canais de temporização; e Lee et al. (2024) demonstram a exploração de metadados associados às políticas de substituição [Xiong et al. 2021, Giner et al. 2024, Hu and Lee 2024, Deng et al. 2019, Lee et al. 2024].

Em termos metodológicos, também existem diferenças relevantes. Os trabalhos utilizam desde experimentos em hardware real, como Xiong et al. e Lee et al., até simulações e modelos, como Hu e Lee e Deng et al. Já Giner et al. exploram um cenário de ataque mais restrito, utilizando um navegador e uma GPU. Em conjunto, essas abordagens mostram que diferentes componentes e níveis da hierarquia de cache podem apresentar comportamentos relevantes para a segurança [Xiong et al. 2021, Giner et al. 2024, Hu and Lee 2024, Deng et al. 2019, Lee et al. 2024].

Nesse contexto, o presente trabalho se posiciona a partir das lacunas identificadas no estudo de Xiong et al. (2021), em particular a pouca cobertura experimental dos níveis superiores da hierarquia. O estudo avalia as políticas LRU e Tree-PLRU na cache L2 por simulação, buscando verificar se as diferenças entre as políticas produzem impactos mensuráveis no comportamento da hierarquia de memória. Aspectos como a LLC, microarquiteturas mais recentes, cenários AMD sem memória compartilhada e mecanismos de randomização permanecem como trabalhos futuros [Xiong et al. 2021].

Assim, diferentemente dos trabalhos que apresentam uma técnica específica de ataque, uma defesa ou uma metodologia de análise, este trabalho realiza uma avaliação experimental das políticas de substituição LRU e Tree-PLRU na cache L2, verificando em que condições elas apresentam diferenças de desempenho.

3. Metodologia

A metodologia experimental foi definida com o objetivo de avaliar se diferentes políticas de substituição de cache influenciam o comportamento e o desempenho da hierarquia de memória. Para isso, foram comparadas as políticas LRU (*Least Recently Used*) e Tree-PLRU (*Tree-based Pseudo-LRU*) sob as mesmas condições arquiteturais e de execução.

A avaliação foi realizada considerando principalmente o desempenho da hierarquia de memória, utilizando métricas como *cache misses*, latência, número de substituições e tempo de execução.

3.1. Simulador utilizado

O simulador escolhido para este trabalho é o gem5, simulador arquitetural amplamente utilizado em pesquisas de sistemas de memória e hierarquia de cache. Ele foi escolhido por dois motivos principais:

- É o simulador utilizado no artigo-base (*Leaking Information Through Cache LRU States in Commercial Processors and Secure Caches*), o que facilita a comparação com os resultados apresentados no estudo original;
- Permite configuração detalhada e customizável de políticas de substituição de cache, necessária para comparar LRU e Tree-PLRU sob as mesmas condições arquiteturais.

3.2. Arquitetura simulada

A arquitetura experimental é composta por um processador de núcleo único.

A hierarquia de memória é composta por caches L1 de instruções e dados e por um nível de cache compartilhado, representado pela L2. A L2 constitui o principal objeto da comparação entre as políticas de substituição.

A configuração inicial foi definida conforme a Tabela 1.

Tabela 1. Configuração arquitetural utilizada nos experimentos.

Componente	Parâmetro	Configuração
Processador	Núcleos	1
	Frequência	2 GHz
	Processos em execução	2
Cache L1 (Instruções)	Capacidade	32 KB
	Associatividade	2-way
	Latência	2 ciclos
Cache L1 (Dados)	Capacidade	32 KB
	Associatividade	2-way
	Latência	2 ciclos
Cache L2 (Compartilhada)	Capacidade	256 KB
	Associatividade	8-way
	Latência	10 ciclos
	Política de substituição	LRU/Tree-PLRU
Memória principal	Tecnologia	DDR4
	Latência	100 ns
Geral	Tamanho da linha de cache	64 bytes

3.3. Variáveis

3.3.1. Variável independente

A principal variável independente é a política de substituição da cache, assumindo dois valores: LRU e Tree-PLRU.

3.3.2. Variáveis dependentes

As principais métricas utilizadas para avaliar o desempenho da hierarquia de memória foram:

- Taxa de cache miss (*miss rate*);
- Número de cache hits;
- Número de cache misses;
- Número de substituições (*replacements*);
- MPKI (*Misses Per Kilo Instructions*);
- Latência média de miss;
- Número de ciclos;
- CPI (*Cycles Per Instruction*);
- Tempo total de execução;
- Tráfego e consumo de energia da memória DRAM;
- Ocupação média da cache L2.

3.4. Workload

Foi utilizado um workload representativo com uso intensivo de memória para avaliar o impacto de cada política de substituição no comportamento da hierarquia de memória.

O mesmo workload foi executado utilizando as políticas LRU e Tree-PLRU, mantendo-se inalterados os demais parâmetros da arquitetura. Dessa forma, foi possível comparar diretamente os resultados e verificar se a política de substituição produziu alguma alteração mensurável no desempenho.

3.5. Estratégia de análise dos resultados

A análise dos resultados foi realizada por meio da comparação entre as políticas de substituição LRU e Tree-PLRU, mantendo-se as mesmas condições experimentais para ambas. Dessa forma, busca-se isolar o efeito da política de substituição sobre o desempenho da hierarquia de memória.

Inicialmente, o workload foi executado utilizando a política LRU, considerada como baseline. As métricas definidas foram coletadas e armazenadas. Em seguida, o mesmo workload foi executado utilizando a política Tree-PLRU, mantendo-se inalterados os demais parâmetros da arquitetura e do experimento.

A sequência geral da análise foi:

1. executar o workload utilizando a política LRU;
2. registrar as métricas obtidas;
3. executar o mesmo workload utilizando a política Tree-PLRU;

4. registrar as mesmas métricas;
5. calcular a variação percentual das métricas em relação ao baseline;
6. comparar LRU e Tree-PLRU quanto às métricas de desempenho.

Para cada métrica, foram calculados os valores absolutos e a variação percentual em relação ao baseline (LRU). Desvios padrão não foram calculados, pois cada configuração foi executada uma única vez.

Dessa forma, a avaliação concentra-se exclusivamente no impacto da política de substituição sobre o comportamento da hierarquia de memória.

4. Ambiente Experimental

Os experimentos foram executados no simulador gem5 em modo de emulação de sistema, com a arquitetura descrita na Tabela 1: um núcleo a 2 GHz, caches L1 de instruções e de dados de 32 KB (2-way), cache L2 de 256 KB (8-way), linhas de 64 bytes e memória principal DDR4.

A única variável alterada entre as execuções foi a política de substituição da L2, configurada como LRU (baseline) ou Tree-PLRU.

As estatísticas foram extraídas do arquivo `stats.txt` gerado pelo gem5 e consolidadas por um *script* de análise, que calcula as variações percentuais em relação ao baseline.

Foram analisadas duas configurações:

- **LRU**: workload de referência com política LRU na L2;
- **Tree-PLRU**: o mesmo workload, com política Tree-PLRU na L2.

As métricas coletadas foram: instruções simuladas, ciclos, CPI/IPC, tempo simulado, acessos, *hits*, *misses*, *miss rate*, MPKI, latência média de *miss* e *replacements* de cada nível de cache, tráfego e largura de banda da DRAM, ocupação média da L2, energia total da DRAM e o mix de instruções emitidas.

Como o gem5 é determinista para uma mesma configuração e semente, cada configuração foi executada uma vez, e por isso não são reportados desvios padrão.

5. Resultados e Discussão

5.1. Impacto da política de substituição (LRU vs. Tree-PLRU)

A Tabela 2 compara as duas políticas. Todas as principais métricas de desempenho foram idênticas: o mesmo número de instruções (248.029) e de ciclos ($2,589 \times 10^6$), o mesmo CPI (10,44) e os mesmos *hits* e *misses* em todos os níveis.

Apenas pequenas diferenças foram observadas na ocupação média da L2 e na energia total da DRAM, ambas de aproximadamente $-0,01\%$. Essas diferenças são desprezíveis e não produziram qualquer reflexo observável no desempenho geral da execução.

A explicação está no padrão de acesso do workload. A L2 teve *miss rate* de 99,54%, com apenas 83 *hits* em 18.031 acessos, ou seja, os dados quase nunca são reutilizados enquanto estão na cache. Nessa situação, a escolha da vítima é pouco relevante:

Tabela 2. Comparação entre LRU e Tree-PLRU na L2.

Métrica	LRU	Tree-PLRU	Δ
Ciclos	$2,589 \times 10^6$	$2,589 \times 10^6$	0,00%
CPI	10,44	10,44	0,00%
Miss rate L1D (%)	10,71	10,71	0,00%
Miss rate L2 (%)	99,54	99,54	0,00%
Hits L2	83	83	0,00%
Misses L2	17.948	17.948	0,00%
Replacements L2	1.564	1.564	0,00%
MPKI L2	72,36	72,36	0,00%
Latência de miss L2 (ticks)	85.928	85.928	0,00%
Leituras DRAM	17.948	17.948	0,00%
Ocupação média L2 (%)	47,7728	47,7696	-0,01%
Energia DRAM (pJ)	$2,741 \times 10^9$	$2,741 \times 10^9$	-0,01%

uma linha removida pela LRU ou pela Tree-PLRU dificilmente terá oportunidade de produzir um *hit* posteriormente.

Políticas de substituição tendem a apresentar diferenças mais evidentes quando existe reuso suficiente para que a decisão sobre qual linha deve permanecer na cache altere a sequência de *hits* e *misses*. Como esse comportamento não ocorre de maneira significativa no workload avaliado, as duas políticas apresentam resultados praticamente indistinguíveis.

O mix de instruções reforça essa interpretação: dominam as escritas de ponto flutuante (131.626 operações `FloatMemWrite` contra 10.649 `MemWrite`), típicas de um preenchimento sequencial de grandes estruturas. Esse padrão de *streaming* produz 16.197 *writebacks* na L1 de dados, além de uma elevada taxa de *miss* na L2.

Além disso, o CPI elevado (10,44) indica que o processador permanece uma parcela significativa do tempo aguardando operações relacionadas à hierarquia de memória, cuja latência média de *miss* fica em torno de $8,6 \times 10^4$ ticks.

Como a Tree-PLRU não alterou nenhuma das principais métricas avaliadas, não houve ganho nem perda de desempenho em relação à LRU para o workload utilizado.

5.2. Relação com a hipótese

A hipótese inicial considerava que a política Tree-PLRU poderia apresentar comportamento de desempenho diferente da LRU devido à sua representação aproximada do estado de utilização das linhas.

Os resultados obtidos não confirmam essa hipótese para o workload utilizado. Não houve diferença significativa entre as políticas em ciclos, CPI, *miss rate*, *hits*, *misses*, *replacements* ou latência de miss.

A explicação mais provável está no padrão de acesso utilizado. Com uma taxa de *miss* de 99,54% na L2, há pouco reuso dos dados armazenados, reduzindo a influência da escolha da vítima sobre os acessos posteriores.

Portanto, os resultados indicam que, para workloads predominantemente sequenciais e com baixo reuso, a substituição aproximada realizada pela Tree-PLRU pode apresentar comportamento de desempenho equivalente ao da LRU.

5.3. Limitações

- **Workload com pouco reuso:** a taxa de *miss* de 99,54% na L2 reduz a capacidade do experimento de distinguir as políticas de substituição.
- **Execução única:** não foram reportados desvios padrão. Em simulação determinista isso afeta menos a análise, mas variações de semente ou de *layout* de memória não foram exploradas.
- **Escopo arquitetural:** foi utilizado apenas um núcleo e a análise concentrou-se na L2, sem avaliação da LLC ou de cenários multi-núcleo.
- **Avaliação restrita ao desempenho:** o estudo não realizou uma avaliação experimental direta da capacidade de exploração do estado de substituição como canal lateral. Dessa forma, as conclusões apresentadas concentram-se no comportamento da hierarquia de memória.

6. Conclusões e Trabalhos Futuros

Este trabalho comparou, por simulação no gem5, as políticas de substituição LRU e Tree-PLRU na L2 quanto ao desempenho da hierarquia de memória, tendo como motivação a relação entre mecanismos de substituição e canais laterais baseados no estado da cache.

Para o workload avaliado, as duas políticas produziram resultados praticamente idênticos em ciclos, CPI, *hits*, *misses*, *miss rate*, *replacements* e latência. A taxa de *miss* da L2, de 99,5%, indica que os dados possuem pouco reuso durante sua permanência na cache. Dessa forma, a escolha da vítima realizada pela política de substituição possui pouca influência sobre os acessos posteriores.

As pequenas diferenças observadas na ocupação média da L2 e na energia da DRAM, ambas próximas de 0,01%, não foram suficientes para produzir impacto mensurável no desempenho da execução.

Os resultados, portanto, indicam que a Tree-PLRU apresenta comportamento equivalente ao da LRU para o workload avaliado. Entretanto, esse resultado não permite concluir que as duas políticas possuem comportamento equivalente para todos os tipos de aplicação, uma vez que workloads com maior localidade temporal e maior pressão sobre os conjuntos da cache podem produzir diferenças mais significativas.

Como contribuição, o estudo documenta o ambiente de simulação e evidencia a importância da escolha do workload em experimentos de comparação de políticas de substituição. Um workload predominantemente sequencial e com baixo reuso apresenta pouca capacidade de revelar diferenças entre políticas que atuam justamente na seleção da próxima linha a ser removida.

Como trabalhos futuros, propõe-se:

- utilizar workloads com maior localidade temporal e *working set* comparável à capacidade da L2, de forma a aumentar a influência da política de substituição;
- avaliar diferentes capacidades e associatividades da L2, permitindo verificar em quais configurações as diferenças entre LRU e Tree-PLRU se tornam mais evidentes;

- estender a análise à LLC e a cenários multi-núcleo, em linha com as lacunas apontadas em Xiong et al. [Xiong et al. 2021];
- avaliar diferentes políticas de substituição além de LRU e Tree-PLRU, permitindo uma comparação mais ampla entre estratégias exatas, aproximadas e aleatórias;
- repetir os experimentos variando semente e *layout* de memória para obter médias e desvios padrão;
- posteriormente investigar experimentalmente a relação entre o estado das políticas de substituição e canais laterais de temporização, utilizando workloads especificamente desenvolvidos para essa finalidade.

Referências

- Deng, S., Xiong, W., and Szefer, J. (2019). Analysis of secure caches using a three-step model for timing-based attacks. *Journal of Hardware and Systems Security*, 3:397–425.
- Giner, L., Czerny, R., Gruber, C., Rauscher, F., Kogler, A., Braga, D. D. A., and Gruss, D. (2024). Generic and automated drive-by gpu cache attacks from the browser. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, pages 128–140. ACM.
- Hu, G. and Lee, R. B. (2024). Random and safe cache architecture to defeat cache timing attacks.
- Lee, J., Sang, F., and Kim, T. (2024). Prime+retouch: When cache is locked and leaked. In *arXiv preprint arXiv:2402.15425*.
- Xiong, W., Katzenbeisser, S., and Szefer, J. (2021). Leaking information through cache hit states in commercial processors and secure caches. *IEEE Transactions on Computers*.